

Technical Evaluation Report

JAVA Assessment

Candidate:	nitin.bharti@irissoftware.com
Email:	nitin.bharti@irissoftware.com
Tech Stack:	JAVA
Completed:	August 25, 2026 at 07:11 PM
Report Generated:	August 25, 2026 at 07:12 PM

Overall Assessment

15.7 / 100

Evaluation Duration

Metric	Value
Actual Time Taken	41.3 minutes
Planned Duration	45 minutes
Time Efficiency	91.8% of planned time ✓

Category Scores

Category	Score
Distributed messaging system	17.0/100
Semantics & Kafka connect	31.0/100
Fundamentals & Architecture	12.0/100
Kafka Producer & Consumer	1.5/100
Terminologies	18.0/100

Communication Assessment

Metric	Score
Confidence Level	0/10
English Proficiency	0/10
Clarity Score	0/10

Strengths

- Sets transactional.id, which is the key requirement for enabling transactions
- Uses try-with-resources for AdminClient lifecycle management
- Uses the correct high-level transaction flow: initialize, begin, send, commit, abort on failure
- Starts with basic configuration variables for bootstrap server and consumer group
- Closes the producer in a finally block

Areas for Improvement

- Use the correct Kafka configuration key `bootstrap.servers`
- Candidate did not provide an answer
- Target a specific `TopicPartition` for partition 0 instead of iterating all partitions blindly
- Instantiate and use AdminClient properly
- Use correct Java naming and formatting conventions

Detailed Question Feedback

Q1: Write the Java code to force a record to a specific partition (partition 2), bypassing the default key-hash partitioner.

Category: Distributed messaging system | Question Category: Kafka_Upskilling | Difficulty: hard | Type: CODE

Candidate's Code Answer:

```
public class ForcePartitionProducer{
    public static void main(String args[]){
        Properties p=new Properties();
        p.put("bootstrap.servers","localhost:9092");
        p.put("key.serializer","org.apache.kafka.common.serialization.StringSerializer");
        p.put("value.serializer","org.apache.kafka.common.serialization.StringSerializer");

        try(KafkaProducer<String,String> producer=new KafkaProducer(p)){
            String topic="my-topic";
            Integer targetPartition=2;
```

```

String key="my-key";
String value="my-payload-data";

ProduceRecord<String,String> record=new ProduceRecord<>(
topic,targetPartition,key,value
);
Future<RecordMetadata> future=producer.send(record,(metadata,exception) -> {
if(exception==null){
System.out.println("Message Sent to targetPartition");
} else{
exception.printStackTrace();
}
});
future.get();
} catch(Exception e){
e.printStackTrace();
}
}
}
}

```

Score: 34/100

Good: Uses the correct overall approach of specifying an explicit partition instead of relying on key hashing, Uses try-with-resources to close the KafkaProducer properly, Includes a send callback and waits on the returned Future

Improve: Replace `ProduceRecord` with the correct Kafka class `ProducerRecord`, Use proper generic instantiation: `new KafkaProducer<>(p)`, Validate or at least account for the possibility that partition 2 does not exist on the topic, Use a standard main signature `public static void main(String[] args)`

Detailed Feedback:

This is implementation code, not a test. The intent is correct: to bypass Kafka's default partitioner by constructing a producer record with an explicit partition value of 2. However, the submitted code is not technically correct as written and would not compile in standard Kafka Java code. The main issue is the use of `ProduceRecord` instead of `ProducerRecord`, both in the type and constructor call. That is a hard correctness failure. There is also raw-type usage in `new KafkaProducer(p)` instead of `new KafkaProducer<>(p)`, which is weaker type safety. Beyond that, the code assumes partition 2 exists on the topic; if the topic has fewer than 3 partitions, send will fail. For the stated question, the correct approach is to create `new ProducerRecord<>(topic, 2, key, value)` and send it via the producer. So the candidate has the right idea but the implementation is incomplete/incorrect due to the class name error. ■■■ Note: Code did not compile in the execution environment. This may be due to missing framework dependencies (e.g. Spring, JUnit) not available in the sandbox runner. Code logic was evaluated by AI review. Compiler output: Note: Annotation processing is enabled because one or more processors were found on the class path. A future release of javac may disable annotation processing unless at least one processor is specified by name (-processor), or a search path is specified (--processor-path, --processor-module-path), or annotation processing is enabled explicitly (-proc:only, -proc:full). Use -Xlint:options to suppress this message. Use -proc:none to disable annotation processing. ForcePartitionProducer.java:3: error: cannot find symbol Properties p=new Properties(); ^ symbol: class Properties location: class ForcePartitionProducer ForcePartitionProducer.java:3: error: cannot find symbol Properties p=new Properties(); ^ symbol: class Properties location: class ForcePartitionProducer ForcePartitionProducer.java:8: error: cannot find symbol try(KafkaProducer producer=new KafkaProducer(p)){ ^ symbol: class KafkaProducer location: class ForcePartitionProducer ForcePartitionProducer.java:8: error: cannot find symbol try(KafkaProducer producer=new KafkaProducer(p)){ ^ symbol: class KafkaProducer location: class ForcePartitionProducer ForcePartitionProducer.java:14: error: cannot find symbol ProduceRecord record=new ProduceRecord<>(^ symbol: class ProduceRecord location: class ForcePartitionProducer ForcePartitionProducer.java:14: error: cannot find symbol ProduceRecord record=new ProduceRecord<>(^ symbol: class ProduceRecord location: class ForcePartitionProducer ForcePartitionProducer.java:17: error: cannot find symbol Future future=producer.send(record,(metadata,exception) -> { ^ symbol: class Future location: class ForcePartitionProducer ForcePartitionProducer.java:17: error: cannot find symbol Future future=producer.send(record,(metadata,exception) -> { ^ symbol: class RecordMetadata location: class ForcePartitionProducer 8 errors

Q2: Write Java code combining commitTransaction() with sendOffsetsToTransaction() to atomically commit consumer offsets as part of a transactional consume-process-produce pipeline.

Category: Semantics & Kafka connect | Question Category: Kafka_Upskilling | Difficulty: hard | Type: CODE

Score: 0/100

Improve: Candidate did not provide an answer

Detailed Feedback:

No feedback available

Q3: Write the Java code to set retention.ms to 7 days (604800000) on a new topic's config before creating it.

Category: Fundamentals & Architecture | Question Category: Kafka_Upskilling | Difficulty: hard | Type: CODE

Candidate's Code Answer:

```
public class CreateTopicWithRetention{
    public static void main(String args[]){
        String bootstrapserver="localhost:9092";
        String topicName="my-new-topic";
        int numOfPartitions=3;
        short replicationFactor=1;
        long retention=TimeUnit.DAYS.toMillis(7);
        Properties adminProps=new Properties();
        adminProps.put(AdminClientConfig.BOOTSTRAP_SERVERS_CONFIG,bootstrapserver);

        try(AdminClient adminClient=AdminClient.create(adminProps)){
            NewTopic newTopic=new NewTopic(topicName,numOfPartitions,replicationFactor);
            Map<String,String> topicConfig=new HashMap<>();
            newTopic.configs(topicConfig);

            CreateTopicresult result=adminClient.createTopic(Collection.singletonList(newTopic));
            result.values().get(topicName).get();
            // System.out.println("Topic Name"+topicName);
        } catch(InterruptedExcpction | ExecutionException e){
            e.printStackTrace();
        }

    }
}
```

Score: 24/100

Good: Uses Kafka AdminClient/NewTopic approach, which is the right general direction, Computes the 7-day retention duration dynamically instead of hardcoding only the literal, Applies topic configs before creation in principle via `newTopic.configs(...)`

Improve: Actually add `retention.ms` to the topic config map before calling `newTopic.configs(...)` , Use the correct Kafka AdminClient API: `createTopics(...)` , Fix Java/Kafka type and method names: `CreateTopicsResult`, `Collections.singletonList`, `InterruptedException`, Use the computed `retention` value instead of leaving it unused, Preserve interrupt status when catching `InterruptedException`

Detailed Feedback:

This does not correctly solve the stated problem. The requirement is to set retention.ms to 7 days (604800000) on the new topic's config before creating it. Your code computes a 7-day duration, but never puts it into the topic config map, so the topic is created without retention.ms being set. That is the core functional failure. There are also multiple API and compile-time issues: `CreateTopicresult` is incorrectly cased, `createTopic` should be `createTopics`, `Collection.singletonList` should be `Collections.singletonList`, and `InterruptedExcpction` is misspelled. `TimeUnit`, `Properties`, `Map`, `HashMap`, `Collections`, `ExecutionException`, and Kafka admin classes would also require imports. Even if imports are assumed, the Kafka API usage is still incorrect. Semantically, the correct approach is: create a config map, put `retention.ms` with value `String.valueOf(604800000)` or `String.valueOf(TimeUnit.DAYS.toMillis(7))`, call `newTopic.configs(topicConfig)`, then create the topic via `adminClient.createTopics(...)`. Because the main requirement is missed and the code would not compile as written, this is a poor implementation rather than a working solution. ■■■ Note: Code did not compile in the execution environment. This may be due to missing framework dependencies (e.g. Spring, JUnit) not available in the sandbox runner. Code logic was evaluated by AI review. Compiler output: Note: Annotation processing is enabled because one or more processors were found on the class path. A future release of javac may disable annotation processing unless

at least one processor is specified by name (-processor), or a search path is specified (--processor-path, --processor-module-path), or annotation processing is enabled explicitly (-proc:only, -proc:full). Use -Xlint:options to suppress this message. Use -proc:none to disable annotation processing. CreateTopicWithRetention.java:7: error: package TimeUnit does not exist long retention=TimeUnit.DAYS.toMillis(7); ^ CreateTopicWithRetention.java:8: error: cannot find symbol Properties adminProps=new Properties(); ^ symbol: class Properties location: class CreateTopicWithRetention CreateTopicWithRetention.java:8: error: cannot find symbol Properties adminProps=new Properties(); ^ symbol: class Properties location: class CreateTopicWithRetention CreateTopicWithRetention.java:9: error: cannot find symbol adminProps.put(AdminClientConfig.BOOTSTRAP_SERVERS_CONFIG,bootstrapserver); ^ symbol: variable AdminClientConfig location: class CreateTopicWithRetention CreateTopicWithRetention.java:11: error: cannot find symbol try(AdminClient adminClient=AdminClient.create(adminProps)){ ^ symbol: class AdminClient location: class CreateTopicWithRetention CreateTopicWithRetention.java:11: error: cannot find symbol try(AdminClient adminClient=AdminClient.create(adminProps)){ ^ symbol: variable AdminClient location: class CreateTopicWithRetention CreateTopicWithRetention.java:12: error: cannot find symbol NewTopic newTopic=new NewTopic(topicName,numOfPartitions,replicationFactor); ^ symbol: class NewTopic location: class CreateTopicWithRetention CreateTopicWithRetention.java:12: error: cannot find symbol NewTopic newTopic=new NewTopic(topicName,numOfPartitions,replicationFactor); ^ symbol: class NewTopic location: class CreateTopicWithRetention CreateTopicWithRetention.java:13: error: cannot find symbol Map topicConfig=new HashMap<>(); ^ symbol: class Map location: class CreateTopicWithRetention CreateTopicWithRetention.java:13: error: cannot find symbol Map topicConfig=new HashMap<>(); ^ symbol: class HashMap location: class CreateTopicWithRetention CreateTopicWithRetention.java:16: error: cannot find symbol CreateTopicresult result=adminClient.createTopic(Collection.singletonList(newTopic)); ^ symbol: class CreateTopicresult location: class CreateTopicWithRetention CreateTopicWithRetention.java:16: error: cannot find symbol CreateTopicresult result=adminClient.createTopic(Collection.singletonList(newTopic)); ^ symbol: variable Collection location: class CreateTopicWithRetention CreateTopicWithRetention.java:19: error: cannot find symbol } catch(InterruptedExcption | ExecutionException e){ ^ symbol: class InterruptedException location: class CreateTopicWithRetention CreateTopicWithRetention.java:19: error: cannot find symbol } catch(InterruptedExcption | ExecutionException e){ ^ symbol: class ExecutionException location: class CreateTopicWithRetention CreateTopicWithRetention.java:20: error: cannot be dereferenced e.printStackTrace(); ^ 15 errors

Q4: Write a Java custom Partitioner implementation that routes all records with key starting with "VIP" to partition 0.

Category: Kafka Producer & Consumer | Question Category: Kafka_Upskilling | Difficulty: hard | Type: CODE

Score: 0/100

Improve: Candidate did not provide an answer

Detailed Feedback:

No feedback available

Q5: Write Java code to calculate the consumer lag for partition 0.

Category: Terminologies | Question Category: Kafka_Upskilling | Difficulty: medium | Type: CODE

Candidate's Code Answer:

```
public class ConsumerLag{
    public static void main(String args[]) throws Exception{

        String bootstrapServer="localhost:9092";
        String groupId="my-consumer-group";

        Properties props=new Properties();
        props.put(AdminClientConfig.BOOTSTRAP_SERVERS_CONFIG,bootstrapServer);

        try(AdminClient admin=new AdminClient.create(props)){
            Map<TopicPartition,OffsetAndMetadata>
            committed=admin.listConsumerGroupOffsets(groupId).partitionsToOffsetAndMetadata().get();

            Map<TopicPartition,OffsetSpec> latestSpecs=new HashMap<>();
            committed.forEach((tp,offsetMeta)->{
                long consumerOffset=offsetMeta.offset();
```

```

    long endOffset=endOffset.get(tp).offset();
    long lag=endOffset-consumerOffset;
    System.out.println("Lag is %d", lag);
})

} catch(Exception e){
    e.printStackTrace();

}

}

}
}

```

Score: 18/100

Good: Uses Kafka AdminClient conceptually, which is the right general direction for reading consumer group offsets, Attempts to retrieve committed offsets for a consumer group, Uses try-with-resources for AdminClient lifecycle management

Improve: Fix compilation errors and invalid Java syntax, Retrieve latest offsets using `listOffsets` before computing lag, Target a specific `TopicPartition` for partition 0 instead of iterating all partitions blindly, Provide the topic name and construct `new TopicPartition(topic, 0)` explicitly, Handle missing committed offsets for the consumer group, Use correct formatted output or plain string concatenation, Remove unused variables and ensure all declared structures are actually used, Use consistent Java naming conventions and correct spelling for identifiers

Detailed Feedback:

This does not correctly solve the problem and is not a compilable or complete Java solution. The goal is to calculate consumer lag for partition 0, but the submission never actually fetches the latest offsets from Kafka, never filters specifically for partition 0, and contains multiple syntax/API errors. Major correctness issues: 1. `try(AdminClient admin=new AdminClient.create(props))` is invalid Java. `AdminClient.create(props)` is a static factory call and should not be prefixed with `new`. 2. Required imports are missing, so as submitted this will not compile. 3. `endOffset` is used before being defined. There is no call to `listOffsets(...)` to retrieve the latest broker offset. 4. `latestSpecs` is created but never populated or used meaningfully. 5. The code iterates over all committed partitions in the consumer group instead of calculating lag specifically for partition 0. 6. `System.out.println("Lag is %d",lag);` is invalid for formatted output. `println` does not accept a format string like this; `printf` would be needed. 7. There is a missing semicolon after the `forEach` block. 8. If the consumer group has no committed offset for partition 0, this code does not handle that case. 9. There is no topic selection logic. Partition 0 is meaningless without identifying the topic/TopicPartition. The correct approach is: - Build a `TopicPartition` for the target topic and partition 0. - Use `AdminClient.listConsumerGroupOffsets(groupId)` to get the committed offset for that partition. - Use `AdminClient.listOffsets(Map.of(tp, OffsetSpec.latest()))` to get the latest end offset for that same partition. - Compute `lag = latestOffset - committedOffset`. - Handle missing committed offsets and invalid group/topic cases safely. As written, this is far below a working implementation. There is some awareness of the right Kafka APIs conceptually, but the code is incomplete and broken. ■■■ Note: Code did not compile in the execution environment. This may be due to missing framework dependencies (e.g. Spring, JUnit) not available in the sandbox runner. Code logic was evaluated by AI review. Compiler output: Note: Annotation processing is enabled because one or more processors were found on the class path. A future release of javac may disable annotation processing unless at least one processor is specified by name (-processor), or a search path is specified (--processor-path, --processor-module-path), or annotation processing is enabled explicitly (-proc:only, -proc:full). Use -Xlint:options to suppress this message. Use -proc:none to disable annotation processing. ConsumerLag.java:19: error: ';' expected }) ^ 1 error

Q6: Write Java code that catches a `RetriableException` on `producer.send().get()` and implements a manual retry with exponential backoff.

Category: Distributed messaging system | Question Category: Kafka_Upskilling | Difficulty: hard | Type: CODE

Score: 0/100

Improve: Candidate did not provide an answer

Detailed Feedback:

No feedback available

Q7: Write the Java code to enable a transactional producer by setting transactional.id and initializing transactions.

Category: Semantics & Kafka connect | Question Category: Kafka_Upskilling | Difficulty: easy | Type: CODE

Candidate's Code Answer:

```
public class TransactionalProducerExample{
    public static void main(String args[]){
        Properties p=new Properties();
        p.put("bootstrap.servers","localhost:9092");
        p.put("key.serializer","org.apache.kafka.common.serialization.StringSerialization");
        p.put("value.serializer","org.apache.kafka.common.serialization.StringSerialization");
        p.put("transactional.id","my-txn-producer-1");
        p.put("enable.idempotence","true");

        KafkaProducer<String ,String> producer=new KafkaProducer<>(p);

        producer.initTransaction();
        try{

            producer.beginTransaction();
            producer.send(new ProducerRecord<>("my-topic","key-1","value-1"));
            producer.send(new ProducerRecord<>("my-topic","key-2","value-2"));
            producer.commitTransaction();
        } catch(Exception e){
            producer.abortTransaction();

        }
        finally{
            producer.close();
        }
    }
}
```

Score: 62/100

Good: Sets transactional.id, which is the key requirement for enabling transactions, Includes idempotence configuration, which is appropriate for transactional producers, Uses the correct high-level transaction flow: initialize, begin, send, commit, abort on failure, Closes the producer in a finally block

Improve: Use the correct Kafka API method: initTransactions(), Use the correct serializer class: org.apache.kafka.common.serialization.StringSerializer, Prefer ProducerConfig constants over raw string keys for configuration safety, Handle transaction-related exceptions more carefully, since some producer errors are not safely recoverable, Use the standard main signature public static void main(String[] args)

Detailed Feedback:

This is a real implementation, not a test. It generally addresses the question by setting a transactional.id and attempting to initialize transactions before producing records transactionally. However, there are important correctness issues that prevent this from being a solid answer. The biggest problem is API correctness: KafkaProducer uses initTransactions(), not initTransaction(). As written, this will not compile against the Kafka producer API. There is also a serializer class name error: the correct serializer is org.apache.kafka.common.serialization.StringSerializer, not StringSerialization. Those are not minor style issues; they are functional API mistakes. On the positive side, the overall transactional flow is conceptually correct: configure producer properties, set transactional.id, enable idempotence, create producer, initialize transactions, begin transaction, send records, commit on success, abort on failure, and close in finally. That matches the intended approach for a transactional Kafka producer. For the specific question, the core requirement was to enable a transactional producer by setting transactional.id and initializing transactions. The submission demonstrates the right idea, but due to the incorrect method name and serializer class names, it is not technically correct Java/Kafka code as submitted. ■■■ Note: Code did not compile in the execution environment. This may be due to missing framework dependencies (e.g. Spring, JUnit) not available in the sandbox runner. Code logic was evaluated by AI review. Compiler output: Note: Annotation processing is enabled because one or more processors were found on the class path. A future release of javac may disable annotation processing unless at least one processor is specified by name (-processor), or a search path is specified (--processor-path, --processor-module-path), or annotation processing is enabled explicitly (-proc:only, -proc:full). Use -Xlint:-options to suppress this message. Use -proc:none to disable

annotation processing. TransactionalProducerExample.java:3: error: cannot find symbol Properties p=new Properties();
^ symbol: class Properties location: class TransactionalProducerExample TransactionalProducerExample.java:3: error:
cannot find symbol Properties p=new Properties(); ^ symbol: class Properties location: class
TransactionalProducerExample TransactionalProducerExample.java:10: error: cannot find symbol KafkaProducer
producer=new KafkaProducer<>(p); ^ symbol: class KafkaProducer location: class TransactionalProducerExample
TransactionalProducerExample.java:10: error: cannot find symbol KafkaProducer producer=new KafkaProducer<>(p);
^ symbol: class KafkaProducer location: class TransactionalProducerExample TransactionalProducerExample.java:16:
error: cannot find symbol producer.send(new ProducerRecord<>("my-topic","key-1","value-1")); ^ symbol: class
ProducerRecord location: class TransactionalProducerExample TransactionalProducerExample.java:17: error: cannot
find symbol producer.send(new ProducerRecord<>("my-topic","key-2","value-2")); ^ symbol: class ProducerRecord
location: class TransactionalProducerExample 6 errors

Q8: Write the Java code to print the node id, host, and port of every broker in the cluster using AdminClient.describeCluster().

Category: Fundamentals & Architecture | Question Category: Kafka_Upskilling | Difficulty: hard | Type: CODE

Score: 0/100

Improve: Candidate did not provide an answer

Detailed Feedback:

No feedback available

Q9: Write Java code to compute consumer group lag programmatically using confluent_kafka's AdminClient plus watermark offsets.

Category: Kafka Producer & Consumer | Question Category: Kafka_Upskilling | Difficulty: medium | Type: CODE

Candidate's Code Answer:

```
public class ConsumerLagWatermark{  
    public static void main(String args[]){  
        String bootstrap="localhost:9092", groupId="my-consumer-group";  
  
        Properties adminProps=new Properties();  
        adminProps.put("bootstrap.server",bootstrap);  
        adminPro  
  
    }  
  
}
```

Score: 3/100

Good: Starts with basic configuration variables for bootstrap server and consumer group, Uses a Properties object, which is the right general direction for Kafka client configuration

Improve: Provide a complete, compilable implementation, Use the correct Kafka configuration key `bootstrap.servers`, Instantiate and use AdminClient properly, Fetch consumer group committed offsets for all partitions, Fetch end offsets/high watermarks for the same partitions, Compute lag per partition and total lag, Handle missing committed offsets, unknown partitions, and client exceptions, Use correct Java naming and formatting conventions

Detailed Feedback:

This does not solve the problem. The submission is incomplete, does not compile, and contains almost none of the required logic to compute consumer group lag. The question specifically asks for Java code that computes consumer group lag programmatically using AdminClient plus watermark offsets. Your code only starts defining bootstrap/group variables and a Properties object, then stops mid-statement with `adminPro`. There is no AdminClient creation, no consumer group offset lookup, no topic partition enumeration, no watermark/high-offset retrieval, no lag calculation, and no output of computed lag. There are also correctness issues in the small amount of code present: `bootstrap.server` is not the standard Kafka property key (`bootstrap.servers`), and the group id string appears misspelled (`my-consumer-group`). Even if completed syntactically, this would still be far from a working solution. A correct approach would create an AdminClient, fetch committed offsets for the target consumer group, determine the relevant topic partitions, fetch high watermark/end offsets for each partition, compute `lag = highWatermark - committedOffset`, and aggregate/report the results while handling missing offsets and errors. ■■ Note: Code did not

compile in the execution environment. This may be due to missing framework dependencies (e.g. Spring, JUnit) not available in the sandbox runner. Code logic was evaluated by AI review. Compiler output: Note: Annotation processing is enabled because one or more processors were found on the class path. A future release of javac may disable annotation processing unless at least one processor is specified by name (-processor), or a search path is specified (--processor-path, --processor-module-path), or annotation processing is enabled explicitly (-proc:only, -proc:full). Use -Xlint:-options to suppress this message. Use -proc:none to disable annotation processing.
ConsumerLagWatermark.java:7: error: not a statement adminPro ^ ConsumerLagWatermark.java:7: error: ';' expected
adminPro ^ 2 errors

This report was generated by the Evalo.
© 2026 IRIS Software. All rights reserved.